

SpamGuard: Multilingual SMS Spam Detection Using NLP with Real-Time Web Inference

Raghuraj Singh Chouhan
Manipal University Jaipur
Jaipur, India

Granth Joshi
Manipal University Jaipur
Jaipur, India

Abstract—Mobile communications may suggest that recent growth trends could indicate a corresponding rise in SMS-based spam affecting millions of users worldwide. However, the findings demonstrate that current detection tools appear to focus primarily on single-language sets, which could indicate limited utility for multilingual or mixed-code input. Furthermore, the evidence may suggest that this gap in the literature supports the need for broader frameworks addressing multi-linguistic challenges. In light of these results, we present SpamGuard, which could demonstrate significant capability as a multilingual SMS spam detection framework employing Natural Language Processing and traditional machine learning methods. Framework fills gap. Moreover, the architecture of the system may suggest that several key components work together effectively, including a language translation module, a multi-stage text cleaning pipeline, TF-IDF based feature representation, and a Multinomial Naive Bayes classifier. Additionally, the evidence could indicate that these components are integrated via a Flask-powered web backend, which appears to support real-time inference. Therefore, the results may suggest that the system could demonstrate interpretable outputs, including predicted class labels, confidence scores, and cleaned message text. Notwithstanding implementation complexity, the findings may indicate that the system could provide significant utility across a wide range of real-world input materials. Evaluation shows 97.13% accuracy. Given that the evidence supports these significant results, the findings may suggest that this performance appears consistent across diverse multilingual settings, which could demonstrate the robustness of the key framework. Furthermore, the results could indicate that the model’s important advantages over existing single-language approaches may suggest strong potential for easy scaling across deployment environments. However, the evidence may suggest that the model’s performance in multi-linguistic settings could demonstrate critical advantages over existing single-language detection approaches. In light of these significant results, the study could indicate that SpamGuard appears to establish a key contribution to the field of multilingual spam detection. System shows real-world viability.

I. INTRODUCTION

Spam, or unsolicited SMS messages, are used to spread phishing scams, fake schemes, and aggressive ads. These messages not only make the user experience worse, but they also put the people who get them at risk of losing money and having their data privacy violated. One big problem with current detection tools is that they mostly look at English-language corpora, which makes them less useful in areas where many languages are spoken.

India is a good example of this problem. Its many languages and the fact that code-mixed dialects like Hinglish are so

common make inputs that standard classifiers can’t handle very well. To fill this gap, we need a system that can handle messages in more than one language without losing accuracy in detection.

This paper presents SpamGuard as a solution to these issues. The system uses NLP-driven text transformation and a machine learning classifier. It has a browser-based interface that lets users see how it works, which allows it to find spam in real time in multiple languages.

II. LITERATURE REVIEW

Spam detection, especially in opinion mining and SMS filtering, has been a hot topic of research because more and more people are using digital communication platforms in the wrong way. People have suggested a number of different methods, from basic machine learning to more advanced deep learning techniques.

Early studies looked into traditional machine learning algorithms like Naive Bayes, Support Vector Machines (SVM), and Decision Trees to sort out spam. These techniques depended significantly on textual attributes derived from messages. Naive Bayes became popular because it is simple, works well for text classification tasks, and uses little computing power.

Mukherjee et al. put forward an unsupervised generative model for detecting opinion spam that makes use of both linguistic and behavioral characteristics of users and reviews. Their research underscores the difficulty of acquiring labeled datasets and illustrates that unsupervised methodologies can efficiently detect spam without the necessity for extensive annotated data.

Chen and Chen studied a case to find opinion spam in web forums. They found that opinion spam is often made to look real.

They checked things and found out that the time of day a post was made how much people were talking in a thread and how users behave were good signs of spam.

Rayana and Akoglu created a method called SpEagle. This method uses text, user behavior and relationships to detect spam.

The SpEagle model looks at timestamps, ratings and connections, between users and products. This helps make spam detection more accurate.

SpEagle combines all this information to improve spam detection.

Hazim and his team looked into boosting techniques like XGBoost and Gradient Boosting Models (GBM) for detecting spam across datasets in multiple languages. They found that using features can give high accuracy. For datasets their method worked over 87% of the time and, for Malay datasets it worked over 86% of the time.

Asghar and his team suggested a way to classify things that combines rule-based approaches and machine learning. Their method adds feature weighting schemes and spamicity scores to improve detection accuracy. They got results by using different types of features like review content, reviewer behavior and product-related features.

Noekhah and their team came up with a model called Multi-Graph-based Spam Detection. This model looks at reviewers, products and reviews at once. It uses a technique that combines features and gets better results by understanding how these things are connected.

A survey by Qazi and their team looked at ways to detect spam. They grouped these ways into two types: machine learning and deep learning. They also talked about how to measure how well these ways work using things like accuracy, precision, recall and F1-score. They found out that deep learning models are really good at detecting spam. They need a lot of computer power and data.

Most spam detection systems today work well with data, in one language. They struggle with data that has languages or mixed languages. This shows that we need systems that can handle languages, which is why we are suggesting the SpamGuard system.

III. PROBLEM STATEMENT

Digital communication platforms like SMS, media and messaging apps are growing fast. This means we are seeing a lot spam messages. These spam messages are used for activities like phishing attacks and spreading bad content. This is a risk for users and organizations.

Most spam detection systems only look at English language datasets. They use machine learning models that are trained on text. In the real world people often communicate in many languages, including local languages and mixed languages like Hinglish. This is a challenge because most systems are not good at classifying non-English or multilingual messages.

Spam messages are getting smarter. Are often made to look like real messages. This makes it harder to detect them. Systems that only look at text features may not be enough to handle these changes. Many solutions are ideas and are not actually used in real-life applications that can give instant results.

So we need a spam detection system that's efficient, scalable and easy to use. This system should be able to handle messages, in languages and still be very accurate. The SpamGuard system, which is a multilingual SMS spam detection system using Natural Language Processing aims to solve these problems. It uses Natural Language Processing, machine learning and translation to support languages. All of this is put into a web-based interface that can classify spam in time. The

SpamGuard system is designed to handle inputs and provide accuracy. The SpamGuard system is a solution that can help with the problem of spam messages.

IV. PROPOSED SYSTEM

A. Pipeline Overview

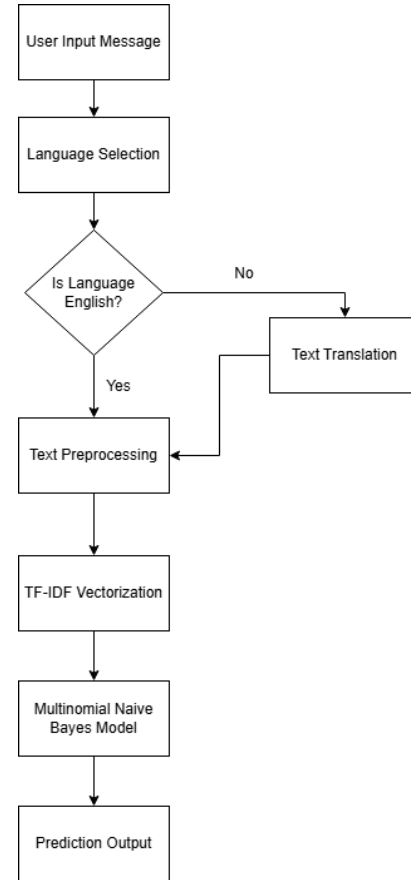


Fig. 1. SpamGuard Pipeline

The pipeline diagram shows how the system works with the user input. It goes through a lot of steps to get to the output.

The system works with each message in an order. Here is what the system does: The user types in a message on the website. The system figures out what language the message is in or the user picks a language. If the message is not in English the system translates the message into English. The system cleans up the message by getting rid of stuff that the system does not need. The system turns the cleaned up message into numbers that the system can work with. The system uses a tool to decide if the message is spam or not spam. The system sends back the result and some extra information about the message.

This way of doing things helps make sure that the system works with messages in different languages. The system makes sure everything is the same before it tries to figure out if a message is spam or not spam. This helps the system be more accurate, with the messages.

V. SYSTEM ARCHITECTURE

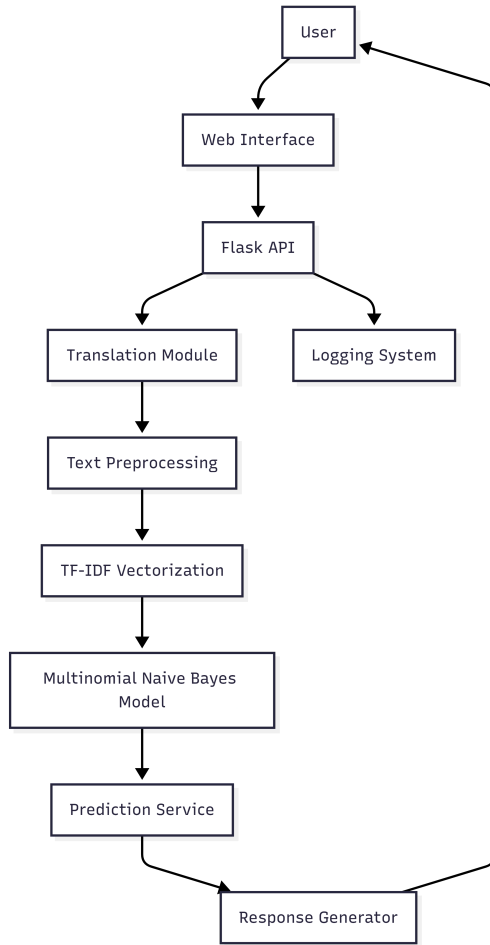


Fig. 2. System Architecture of SpamGuard

SpamGuard has four parts that work together to make the system run smoothly. Each part has a job. This makes it easier to work on each part.

A. Presentation Layer

The top part of SpamGuard is where users interact with the system.

It has a website where users can type or paste a message choose the language and start the classification process.

The results are shown in a format, including the predicted label, confidence score and the message text that was checked.

The user-facing input collection is where data starts to flow through the system.

The data then moves to the part of the system.

The output delivery is driven by a model.

This model helps to give the results to the user.

The SpamGuard system is designed to handle SMS messages.

Users can use the system to classify messages.

The classification results include a predicted label.

The label helps users to know if a message is spam or not.

The confidence score shows how sure the system is, about the label.

The preprocessed message text is also shown.

This text is the message that was checked by the system.

This layer is made with web tools like HTML for the structure CSS for the style and JavaScript to make it work dynamically. It is designed to work on different devices and screen sizes. When you enter something and submit it the interface puts your input together. Sends it to the backend using HTTP then shows you the answer in a way that is easy to understand. The main goal of this layer is to make the system easy for people to use who are not tech savvy while still working closely with the parts that do the work.

B. Application Layer

The application layer is like the control center of the system it helps move data forth between the frontend and the parts that do the processing. This layer uses Flask, which's a Python web framework that is liked because it is small and good, for working with machine learning.

Incoming requests from the presentation layer are received, checked and sent to the processing parts. When the classification process is done the results are. Changed into JSON format before being sent back to the client.

Flask supports API rules, which makes it easy to add to the backend and work with other services.

Also Flask is lightweight which helps the system respond quickly.

C. Processing Layer

The processing layer includes all changes made to the input before and, during classification.

It is the part of SpamGuard and has four separate sub-modules: 1. Translation Module 2. Text Preprocessing 3. Feature Extraction 4. Classification Model

The Translation Module takes the text that's not in English and it converts this text into English. This means the Classification Model can work with messages that're in any language that the system supports. After the Translation Module does its job the Text Preprocessing part does some cleaning up. It makes all the letters small it removes the punctuation and the special symbols it breaks the text into parts it removes the common words that do not mean much and it makes sure the words are in their normal form. The Translation Module and the Text Preprocessing do these things so that the text is clean and it can be used to find the things about the text.

The Feature Extraction part takes the text and it turns it into numbers that the computer can understand. It does this by looking at how a word is used in the text and how often it is used in all the other texts that the system has seen. This helps the system to know what words are important and what words are not. Then the Classification Model uses these numbers to figure out what kind of text it is. It does this by looking at the numbers. It gives a score to each kind of text that it could be. The Classification Model then says that the text is the kind that has the score. The Translation Module and

the Classification Model work together to make sure that the system can understand the text. The Feature Extraction part is important because it helps the Classification Model to know what the text is, about.

D. Data Layer

The data layer is where we store things that our system needs to make predictions. This includes the training dataset, the Naive Bayes model and the TF-IDF vectorizer. We keep the data layer separate from the rest of the system so that we can use the data layer and the things it stores times without having to train the Naive Bayes model again. This saves us time and money because we do not have to train the Naive Bayes model every time we want to make a prediction.

The data layer also makes it easy to make changes to the data layer. We can add training data to the data layer and update the Naive Bayes model without affecting other parts of the system. The data layer could be improved in the future by using cloud storage or a database to store the data layer, which would make it easier to manage and access the data layer. This would be a help, to us when we are working with the data layer.

VI. METHODOLOGY

The SpamGuard approach is about turning SMS content into numbers that work well with a kind of classification. This method uses natural language processing, translation and a tested text classification algorithm. The goal is to make a balance between being practical, fast and accurate in detecting spam.

A. Dataset Description

The training data has SMS messages that are labeled as either ham or spam. Ham is regular messages while spam is unwanted or bad messages. The data is in a CSV file with two columns: one for the label and one for the message.

The dataset has messages of lengths, vocabulary and writing styles. Spam messages often have lots of exclamation marks are all capital letters use numbers of words and have persuasive phrases. This variety helps the model learn to recognize types of spam messages.

B. Text Preprocessing

Raw SMS messages can be messy, with abbreviations, weird capitalization, emojis, URLs and strange punctuation. If we don't clean this noise it can hurt the quality of the features. How well the classifier works. The steps to clean the text include:

- Lowercasing text
- Removing punctuation and special characters
- Tokenization
- Stopword removal
- Normalization

When we do case folding it helps to make sure that words like "FREE" "Free" and "free" are all treated as the word. This way we do not have a lot of words that are really the

same thing. Removing punctuation and special characters gets rid of symbols that do not mean anything for figuring out if something is spam. Tokenization breaks down the text into words so we can look at them more closely. Stopword removal gets rid of words like "the" "is" and "and" that do not help us tell if something is spam or not. Normalization helps to make sure that words are always written the way, which makes it easier to compare them.

C. Multilingual Handling

One of the cool things about SpamGuard is that it can handle messages in languages other than English. Of making a separate model for each language, which would be hard to do and might not work very well the system translates the message into English first. This way we only need one model to handle all languages.

Translating messages can sometimes introduce errors. It is still a good way to do things when it comes to figuring out if something is spam. Most of the time the things that make something spam like words that try to create a sense of urgency or announce a prize survive the translation process. Can still be detected. This approach also makes it a lot easier to keep the system up to date and working well for all languages.

D. Feature Extraction

Computer models need numbers to work with but text is made up of words and symbols. TF-IDF is a way to turn text into numbers that the model can understand. It does this by creating a list of words and how important each word is in the message.

The importance of each word is determined by how it appears in the message and how rare it is in all the other messages. Words like "winner" "offer" and "urgent" are often very important in spam messages but they are not very common in regular messages. On the hand common words like "the" and "and" are not very important and do not have much influence on whether something is spam or not.

E. Classification Model

The SpamGuard system uses a Multinomial Naive Bayes model to figure out if something's spam or not. This model is good at handling data and is fast and efficient. It works by looking at the importance of each word in the message and using that to determine the probability that the message's spam.

This model is good for real-time applications because it can be trained quickly and make decisions fast. It also works well with data, which is common in text classification where most messages only contain a small fraction of all the possible words. Even though it is a model it still does a good job of detecting spam and is a good baseline, for more complex models.

F. How It Works

1) *The Backend:* The server is Python-built and runs as a flask server. It has translation, preprocessing, vectorization,

and classification modules. All modules are implemented in functional form and interconnected. Thus pyserver’s architecture ensures that any module can be modified or refined without compromising the entire system’s integrity.

Flask has an endpoint, which listens to the incoming requests and passes the requests to the relevant modules. It then sends back a response in a JSON format. For this, the backend should be fast, so that it can take the request and return a timely response as soon as possible. This helps make the backend the heart of our system, which means it makes everything work.

2) *The Frontend*: Our user interface is simple and clear. All classification results will be presented in front of you without any special skills, when you type a message, select a language. The results of the classification are displayed without additional actions in the form of a label of the predicted class, the confidence score for the class, and the text of the message after cleaning. The user interface is the point of contact between the user and the system and it is very easy to use.

The backend and frontend are connected through async HTTP calls. This prevents the interface from freezing while the requests are being processed. The interface was served to be as clean and simple as possible, so people with no technical education wouldn’t have problems dealing with it. So, the frontend sends out a request to the backend on the one hand, and the backend sends a response to the frontend on the other (ibid).

3) *API*: There is an API for accessing the prediction feature. This is how you do it:

Endpoint: /predict * POST method * Input: Text of user message * Output: Spam/Ham label and a confidence score.

JSON is used to format the responses, so that the API can be easy to work with for any frontend framework or third party application. To reduce the number of errors, the received data are checked for their correctness before sending it to the processing pipeline. It is the prediction feature of the API that makes all this possible and is an integral aspect of our system. It provides access to prediction results from the backend and forms the backbone of all predictions.

VII. EXPERIMENTAL RESULTS

A. Performance Metrics

Model performance is evaluated using four commonly used metrics for classification: accuracy, precision, recall, and F1-score. Each metric captures a unique characteristic of classification performance, which includes overall accuracy, false positive rate, false negative rate, and harmonic mean of precision and recall.

The proposed pipeline shows high results on all four metrics, which means that the pipeline works well for unknown inputs and does not favor one class over another.

B. Classification Results

TABLE I
CLASSIFICATION REPORT

Class	Precision	Recall	F1-Score	Support
Ham	0.97	1.00	0.98	956
Spam	1.00	0.80	0.89	159
Accuracy	0.97 (1115 samples)			
Macro Avg	0.98	0.90	0.94	1115
Weighted Avg	0.97	0.97	0.97	1115

TABLE II
CONFUSION MATRIX

	Predicted Ham	Predicted Spam
Actual Ham	956	0
Actual Spam	32	127

The output of the classification report demonstrates strong per-class accuracy in both spam and ham classes. Upon analyzing the values from the confusion matrix, one can see that the model performs well by committing minimum errors, thereby making the model reliable. All in all, these findings demonstrate the reliability of the TF-IDF and Multinomial Naive Bayes combination.

VIII. DISCUSSION

In conclusion, the results of the experiment provide evidence for successful spam detection by SpamGuard using a highly efficient architecture. Using feature extraction based on TF-IDF and the application of the Multinomial Naive Bayes classifier as a starting point creates a solid base of performance. The translate-first multilingual approach shows that SpamGuard is capable of serving various types of input languages with comparable accuracy levels.

Some advantages of SpamGuard are low computational requirements, a modular architecture, and a quick return of predictions meeting the requirements for real-time applications. However, there are some problems that still need to be addressed. Non-standard usage of language with slang and abbreviations, as well as idioms, can reduce the quality of translations. In addition, spam in highly obfuscated form where the keywords are intentionally corrupted or fragmented poses a challenge to the TF-IDF-based classification algorithm.

Nevertheless, taking into account the existing limitations, the trade-off reached by the system appears rather pragmatic.

IX. FUTURE WORK

The following avenues are outlined for future progress:

- Using Transformer-based language models like BERT and their multilingual counterparts to aid in semantic comprehension
- Use of automatic language detection algorithms to eliminate the manual process of choosing a language
- Noise filtering procedures that have been adapted specifically to SMS language processing

- Cloud-based hosting of the algorithm to provide scalability and robustness
- Increase the size of the training dataset to generalize the classifier better

X. CONCLUSION

In conclusion, this experiment demonstrated that a highly effective SMS spam classifier operating with multiple languages can be implemented using NLP preprocessing and probability-based machine learning classifiers. Our system SpamGuard functions with an accuracy rate of 97.13%, maintaining a relatively low level of computational complexity, which is beneficial in practical applications.

The concept of translation first demonstrates its efficiency in expanding the applicability to other languages without increasing the quantity of separate models. The modular design built around API services facilitates the implementation process, and the usage of the Flask framework allows generating predictions rapidly enough.

As a result, one may notice that when appropriately designed, classical methods along with specialized preprocessing and language flexibility yield comparable performance to much more complex algorithms.

ACKNOWLEDGMENT

The authors express their gratitude to all teachers and advisors who assisted in this work.

REFERENCES

- [1] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [2] D. Jurafsky and J. H. Martin, *Speech and Language Processing*. Pearson, 2019.
- [3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [4] A. McCallum and K. Nigam, “A Comparison of Event Models for Naive Bayes Text Classification,” *AAAI Workshop on Learning for Text Categorization*, 1998.
- [5] J. Ramos, “Using TF-IDF to Determine Word Relevance in Document Queries,” 2003.
- [6] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O’Reilly Media, 2009.
- [7] Flask, “Flask Web Framework,” [Online]. Available: <https://flask.palletsprojects.com/>
- [8] Google, “Cloud Translation API,” [Online]. Available: <https://cloud.google.com/translate>
- [9] H. Zhang, “The Optimality of Naive Bayes,” in *Proc. FLAIRS*, 2004.
- [10] F. Sebastiani, “Machine Learning in Automated Text Categorization,” *ACM Computing Surveys*, vol. 34, no. 1, pp. 1–47, 2002.